SAGA: Schedule Abstraction Graphs at GPU Speed

***Abstract*—Accurate schedulability analyses are essential to the certification of safety-critical real-time systems, but their reliance on exhaustive state-space exploration makes them notoriously expensive. Over the past decade, the Schedule Abstraction Graph (SAG) has shown a tractable path through this barrier with two key ideas: reframing schedulability as the systematic exploration of possible job dispatch orders, and merging semantically equivalent states reached along different orderings. Together, these ideas keep exhaustive enumeration manageable across a growing range of workload models, including non-preemptive uniprocessor jobs, moldable gang tasks, limited-preemptive parallel DAG tasks, and self-suspending tasks with event-driven delays, making the SAG family a natural baseline for the system models it covers. Its precision, however, comes at a cost: existing single-threaded CPU implementations are slow enough to limit the workload sizes researchers and practitioners can analyze in practice.**

We address the SAG bottleneck with techniques from high-performance computing. SAGA is the first CPU-GPU heterogeneous framework that accelerates the SAG family of analyses by restructuring their computation for the GPU. To demonstrate generality across the family, we instantiate the full lineage of published SAG algorithms within it. Across 36 workload configurations, SAGA achieves measured speedups of more than 470× over the corresponding state-of-the-art CPU tools; on larger workloads where existing analyses become intractable, SAGA yields its largest gains, with conservatively extrapolated speedups exceeding three orders of magnitude. Beyond raw speedup, this turns hours-long analyses into seconds-long ones and shifts the baseline against which future schedulability analyses, approximate or otherwise, should be measured.

I. INTRODUCTION

Schedulability analyses underpin the certification of safety-critical real-time systems. Before a system is deployed, a rigorous analysis must confirm that every task will meet its timing constraints under all possible execution scenarios. Because the true worst-case scenario is often unknown due to timing anomalies and parameter variations, simulation-based approaches cannot provide sound guarantees. Alternatively, traditional sufficient schedulability tests (such as classical response-time analysis or demand-bound methods) address this by pessimistically over-approximating the worst case, often rejecting valid task sets to maintain scalability. Conversely, exact analysis avoids this pessimism by exhaustively reasoning about the space of executions a workload can produce, but this exhaustive analysis is notoriously expensive.

Among schedulability analyses, the Schedule Abstraction Graph (SAG) [1] stands out by achieving an elegant balance between tractability and precision. Its central insight is to reframe schedulability as the systematic exploration of possible job dispatch orders: each path in the graph corresponds to one feasible scheduling decision sequence, and each node abstracts the system states reachable along that prefix. A sound merge operation then collapses semantically equivalent states, keeping the exploration tractable without sacrificing precision, and, for several variants, preserving full *exactness*. This paradigm has proven remarkably extensible. Over the past decade, the SAG family has grown from non-preemptive uniprocessor scheduling [1] to global multiprocessor scheduling [2, 3], limited-preemptive parallel DAG tasks [4], event-driven workloads with self-suspensions [5], and time-aware shapers in TSN networks [6], catalyzing a robust ecosystem of analytical extensions and supporting tools. For the system models it covers, the SAG is a natural reference point against which other analyses are measured.

This breadth, however, has come at a real cost: every variant in the SAG family inherits the same single-threaded, sequential exploration of a state graph that grows combinatorially with the number of jobs and the expressiveness of the workload model.[1] While SAG implementations have already pushed three orders of magnitude past comparable model-checking tools such as UPPAAL [4], the community has openly noted that scalability remains a fundamental limitation, particularly under release jitter and execution-time variation [7]. Published analyses of more expressive variants can take minutes to hours, and many runs simply time out before producing a result [4, 5]. The community's response has been entirely *algorithmic* [7, 8]: partial-order reduction, lazy abstraction, sharper state representations, and more aggressive merging. These efforts have produced real gains, but they all operate within the same sequential execution model, ultimately bounded by single-core performance. Despite these algorithmic strides, breaking the sequential bottleneck through new computational paradigms remains a critical and unresolved challenge in the field.

This is the gap SAGA closes. Modern GPUs ship with tens of thousands of compute cores and are standard hardware on the servers where schedulability analysis is actually run. Yet, during analysis, these powerful parallel resources sit idle while a single CPU core does all the work. SAGA shifts the focus from purely algorithmic improvements to algorithm-hardware co-design by fundamentally restructuring the SAG computational paradigm for CPU-GPU heterogeneous systems.

[1]The SAG family is canonically maintained in the SAG-org repository, which tracks four published algorithms: non-preemptive uniprocessor scheduling [1], non-preemptive moldable gang scheduling [3], limited-preemptive global parallel DAG scheduling [4], and limited-preemptive self-suspending scheduling with event-driven delays [5]. Unless otherwise noted, references to the SAG family, the SAG lineage, or SAG algorithms in this paper denote these four results. Other works that build on SAG, e.g., on TSN time-aware shapers [6], are not part of this codebase and are not evaluated here. While the reference implementation includes a CPU thread-parallel build backed by Intel TBB, only the ECRTS 2019 evaluation by Nasri et al. [4] used it; subsequent publications report only single-threaded numbers. A source-level review surfaced concurrency defects that make its outputs non-reproducible across runs, so we compare against the sequential build. That a working parallel implementation has not yet landed even in the canonical codebase reinforces the broader point: parallelizing SAG correctly is non-trivial.

SAGA makes four contributions. **(1)** We design the first CPU-GPU heterogeneous framework for accelerating SAG-style schedulability analyses, parallelizing the expansion and merging of system states on the GPU while keeping the CPU productively engaged. **(2)** We introduce a set of GPU-aware optimizations that further reduce the work the analysis needs to do, without affecting its precision. **(3)** We instantiate the full lineage of published SAG algorithms [1, 3–5] within the framework, demonstrating that SAGA generalizes across the SAG family, spanning non-preemptive uniprocessor, global multiprocessor, limited-preemptive parallel DAG, and event-driven self-suspending workloads. **(4)** We evaluate SAGA against Srinivasan et al.'s SAG-based analysis of limited-preemptive self-suspending tasks with event-driven delays [5] — the most computationally demanding member of the SAG lineage. SAGA achieves measured speedups of more than 470× over the corresponding state-of-the-art CPU tools; on larger workloads where existing analyses become intractable, SAGA yields its largest gains, with conservatively extrapolated speedups exceeding three orders of magnitude.

II. SAG and the Case for Hardware Acceleration

We begin by reviewing the computational paradigm common to all SAG variants, before arguing why its inherent complexity motivates the need for hardware acceleration.

Consider n jobs executing on m identical cores under a global job-level fixed-priority policy. Each job's release time, execution time, and (depending on the model) suspension or delay parameters are given as bounded intervals rather than fixed values. An *execution scenario* is a concrete assignment of values to these parameters; the task set is *schedulable* iff every job meets its deadline under every execution scenario.

The analysis constructs a directed acyclic graph $G = (V, E)$ layer by layer, where each node (a *state*) symbolically represents a set of execution scenarios sharing the same sequence of dispatch decisions. Variants differ in what each state tracks, but every SAG state encodes three categories of information: **(i)** the set of dispatched jobs D; **(ii)** when cores will become available, as a vector $A = (A_1, \ldots, A_m)$ of interval-valued bounds; and **(iii)** when previously dispatched jobs will finish, as a set F of finish-time intervals for jobs whose completion still constrains later dispatches. The graph is built through an alternating sequence of *expansion* and *merging* phases.

For each state on the current layer $\mathcal{L}_i$, the expansion phase identifies every job that could be dispatched next and, for each valid candidate, creates a successor state. The precise eligibility and timing computations differ across workload models, but the structure is invariant: for each (state, candidate) pair, the analysis computes bounds on the earliest and latest possible start and finish times, and constructs a child state with updated dispatch, availability, and finish-time information.

Without countermeasures, the number of states would grow combinatorially with each layer. The merging phase suppresses this explosion by consolidating states that are "similar enough" into a single abstract state. Two states are mergeable if **(C1)** they share the same set of dispatched jobs, **(C2)** their core-availability intervals pairwise overlap, and **(C3)** their finish-time intervals pairwise overlap. The merge operator forms a new state by taking component-wise extrema of all interval bounds. Merging is *sound*: any execution scenario representable by either original state remains representable by the merged state, so no feasible schedule is lost. However, merging may also admit scenarios not present in either original state, and is therefore *exact* for some SAG variants but *pessimistic* for others. Merging is what makes the SAG more tractable, and what sets it apart from simulation and model-checking approaches. Figure 1 illustrates an example.

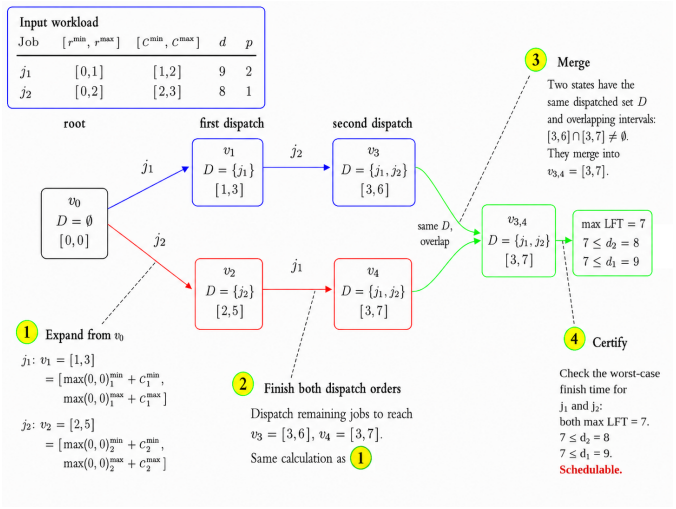

Fig. 1: SAG construction for the example workload. The table lists each job's release-time bounds $[r^{min}, r^{max}]$, execution-time bounds $[C^{min}, C^{max}]$, deadline d, and priority p (smaller p is higher priority). Starting from root v_0, the analysis proceeds layer by layer: each edge dispatches one eligible job, updates the dispatched set D, and computes the finish-time interval $[EFT, LFT]$. The first layer represents the two possible first dispatches, and the next layer completes the two dispatch orders. States with the same D and overlapping intervals, such as v_3 and v_4, are merged into $v_{3,4}$ to curb state growth. The final worst-case finish time satisfies all deadlines, certifying schedulability.

The case for hardware acceleration. Despite the effectiveness of merging, the per-layer state count can still grow exponentially in the number of eligible jobs. At each layer $\mathcal{L}_i$, the expansion phase may produce up to $|\mathcal{L}_i| \cdot |R|$ children, where $|\mathcal{L}_i|$ is the current layer width and $|R|$ is the number of ready candidates. In the worst case, $|R|$ grows as $O(n)$ and the merge compression ratio remains bounded, yielding an exponential lower bound on the total state-space size.

This exponential state-space growth is not merely a theoretical artifact. Alternative exact approaches based on timed-automata model checkers such as UPPAAL [9] do not scale to industrial workload sizes: even for simple non-preemptive periodic tasks, UPPAAL requires an average of 45 minutes to analyze 24 tasks on 4 cores at 30% utilization, with nearly

half of the tested workloads timing out after one hour [4]. The SAG offers vastly superior scalability through its merge operation, but recent measurements show that sequential CPU implementations still require minutes to hours on moderately sized workloads. In one such evaluation, the preemptive SAG required over 12,000 seconds (more than three hours) on a 10-core workload at 60% utilization [8].

At its root, schedulability analysis under release jitter and execution-time variability is computationally hard: the basic uniprocessor case is coNP-hard, and the multiprocessor case is at least as difficult [10, 11]. SAG's exhaustive exploration reflects this underlying complexity. Algorithmic optimizations such as partial-order reduction [7] can collapse redundant branches and yield large speedups in practice, but they cannot alter the worst-case exponential scaling of the state space.

What *can* change is the rate at which this state space is consumed. Two structural properties of the SAG make it amenable to massively parallel execution. First, the expansion of each (state, candidate) pair on a given layer is independent of all other pairs on the same layer, exposing data parallelism that scales directly with layer width. Second, the merging phase operates entirely within a single layer, and merge condition C1 induces a natural partitioning: only states sharing the same set of dispatched jobs D are mergeable, so the search for merge candidates parallelizes over the resulting partitions. These observations motivate the shift from purely algorithmic optimization to hardware-accelerated execution.

III. Fundamental Barriers to a Direct GPU Port

The two structural properties at the close of §II suggest that the SAG implementation can be directly ported to a GPU-aware implementation. Each (state, candidate) pair expands independently of every other within a layer, and once states are grouped by their dispatched set, the search for merge candidates parallelizes across the resulting groups. A canonical implementation would therefore assign one thread per state, enumerate the eligible candidates in parallel, and consolidate each layer through a hash-based merge before advancing.

However, this naive pipeline inevitably collapses. In this section, we expose four fundamental mismatches between how GPUs run code and how the SAG produces work. These obstacles are not mere implementation artifacts that can be resolved by incremental tuning; rather, they are inherent complexities that necessitate a purpose-built framework. We introduce the relevant GPU architectural concepts as needed, referring the reader to [12] for a fuller account.

Uneven work per thread. GPUs run threads in fixed-size groups called *warps* (which consist of 32 threads on NVIDIA hardware), under a Single-Instruction Multiple-Thread (SIMT) model, in which all threads in a warp execute the same instruction at once, each on its own data. When threads take different branches of a conditional, the hardware runs each branch in turn with the off-path threads idle, a phenomenon known as *branch divergence*. A fully divergent warp thus runs up to 32× slower than a perfectly aligned one.

The SAG hands the GPU exactly the workload this model handles worst. Within a single layer, states differ in which jobs they have dispatched and in the finish-time intervals they have reached; one state can therefore expose many ready successors while another exposes only a few. For parallel DAG [4] and self-suspending workloads [5], the gap widens further; two states sharing the same dispatched set can produce entirely different successors, because which jobs can dispatch next depends on when their predecessors finished, not just on which jobs are already dispatched.

Under the obvious one-thread-per-state mapping, the warp inherits this imbalance: 31 of its 32 threads wait while one iterates through its candidates. Each candidate then runs through a chain of conditional tests on predecessors, priorities, start time, and deadline, and each test gives the warp another chance to diverge. In practice, the nominal speedup of moving expansion onto the GPU evaporates.

Unpredictable memory growth. GPUs are built for buffers whose size is known in advance and for *coalesced* access, in which threads in a warp read consecutive addresses so the hardware can fetch their data in a single transaction.

The SAG breaks both expectations. The size of layer $\mathcal{L}_{i+1}$ is not known until layer $\mathcal{L}_i$ has finished expanding and merging: how many successors a (state, candidate) pair produces depends, at run time, on precedence, priority, and interval overlap. The states themselves also lack a uniform memory layout: the finish-time set F grows and shrinks across states, depending on which dispatched jobs still have undispatched successors [4], so adjacent states in memory carry different fields, and a warp reading them in parallel cannot coalesce.

Two obvious workarounds both fail. Allocating memory inside the kernel for each new state is slow, and out-of-memory failures inside a kernel are hard to recover from. Reserving a worst-case buffer for every layer wastes most of it on typical layers and exhausts VRAM long before the state space itself demands it. A workable implementation has to hold both at once: enough capacity for an algorithm whose layers can explode, and a per-layer footprint small enough not to exhaust device memory in the process.

The merge bottleneck. GPUs offer hardware support for the warp of threads to exchange register values with each other in a few cycles, so they can answer questions like "which of us holds a true predicate?" or "what is the smallest value among us?" in a single instruction. Algorithms built on this support are called *warp-cooperative*: they assign one logical task to the entire warp, with the threads each handling a slice and the hardware stitching the slices together. Per-thread work stays uniform; divergence vanishes.

SAG's merge phase resists this pattern. A pairwise compatibility check between two states verifies three conditions from §II: equal dispatched sets (C1), pairwise overlap of all m core-availability intervals (C2), and pairwise overlap of every interval in F (C3). The three conditions touch three different data structures of three different size: a bitmask of length n, a vector over m cores, and a set of $|F|$ intervals whose size

itself varies from state to state. There is no even 32-way split of this work. However the warp distributes it, some threads run short, some run long, and the uniform per-thread workload that warp-cooperative algorithms depend on never materializes.

Existing SAG implementations keep merging tractable by hashing states on their dispatched set D: a new state compares only against the few others in its bucket. But this approach is *inherently sequential*. Every time two states in a bucket merge, the merged state has wider intervals than either input, and whether a later arrival qualifies to merge depends on those widened intervals. Whether state k ends up merged, and with which others, therefore depends on which of the preceding $k-1$ states already merged together, and on how their intervals grew along the way. No parallel ordering of the merges can respect this dependence without serializing the phase. Putting merging on the GPU means reformulating the problem, not porting the sequential algorithm.

The host-device gap. The bandwidth between CPU memory and GPU memory is typically one to two orders of magnitude lower than the bandwidth inside the GPU. The SAG advances layer by layer; shipping each layer's frontier across this bus saturates the interconnect long before the GPU's compute is busy. And many layers, especially at the start and end of the construction, contain too few states to amortize the cost of launching a kernel; on those layers, GPU execution is strictly slower than running everything on the CPU.

Our work. These four obstacles together explain why off-the-shelf GPU frameworks do not accelerate the SAG. Branch divergence degrades throughput during expansion. Dynamic memory growth defeats static allocation strategies and effective memory accesses. The merge phase resists parallel reformulation. The host-device interface penalizes layered synchronization. A practical framework must therefore overcome all four challenges simultaneously. The remaining sections of this paper develop such a framework, Saga.

IV. The Saga Framework Architecture

§III identified four obstacles that any practical GPU-accelerated SAG framework must address. This section describes the orchestration layer of Saga, which resolves two of them at the architectural level: dynamic memory growth and the host-device gap. The algorithmic primitives it invokes, the warp-cooperative expansion kernels of §V and the sort-based merge pipeline of §VI, are pluggable operators: they interact with the architecture only through a small set of GPU-resident interfaces, so a different SAG variant is obtained by substituting them, with the orchestration layer untouched.

Saga treats the SAG as a sequence of *layers*, each processed end-to-end before the next begins, and each decomposed into *waves* of bounded size. A wave is the unit of GPU work. It consists of one kernel launch for eligibility evaluation, one for successor creation, and a short pipeline of kernel launches implementing the per-wave merge. A layer completes when every wave has been processed and its merged states redeposited into the layer-resident buffer. Three orchestration mechanisms

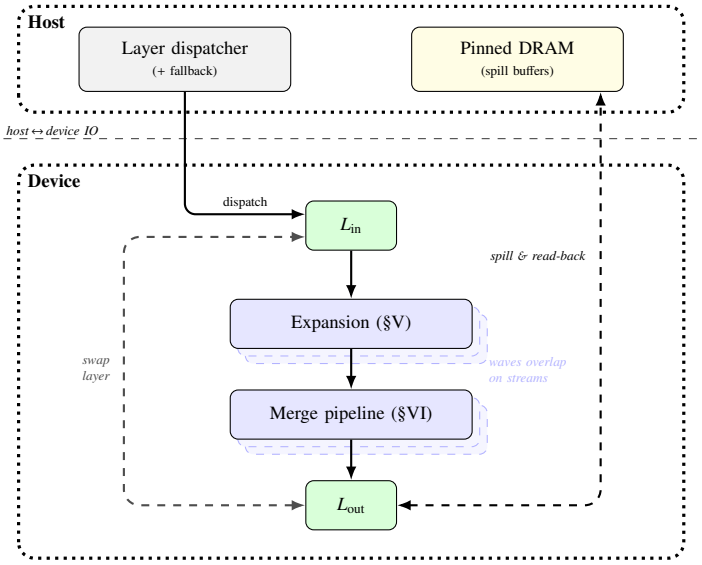

Fig. 2: Framework architecture. The host *layer dispatcher* routes each layer to the GPU or CPU path. On the GPU, two persistent VRAM buffers (L_{in}, L_{out}) alternate roles at each layer boundary. Within a layer, every wave traverses two compound stages: *Expansion* (§V) and the *Merge pipeline* (§VI). The two stages run on CUDA streams so that consecutive waves overlap. Pinned DRAM *spill buffers* absorb VRAM overflow during layer production and supply read-back during the consumption of the next layer; the host–device interconnect otherwise carries only small per-layer metadata.

span this structure: streaming with GPU persistence, layer-residency management with host-side overflow, and adaptive CPU fallback for narrow layers. Figure 2 shows these three mechanisms working in concert around the wave pipeline of a single layer.

Wave-based streaming with GPU persistence. The layer-at-a-time design examined in §III would move each frontier across the host-device interconnect twice per iteration. Saga avoids this by keeping the entire layer resident in a pair of GPU-persistent buffers that alternate as input and output across consecutive layers. Within a layer, waves read a contiguous slice of the input buffer, produce expanded states into per-wave scratch regions, and deposit the merged result into the buffer of the next layer. During ordinary layer processing, no state data crosses the host-device interconnect; the only traffic is small per-layer metadata: the wave merge counts and the final response-time bounds.

Two CUDA streams run consecutive waves in a double-buffered schedule: while one stream computes a wave, the other prefetches the input of the next wave and retires the output of the previous wave. Inter-wave data movement is thereby overlapped with compute, and the kernels stay continuously fed.

Layer residency and host-side overflow. VRAM is finite, and the number of states in a single layer can exceed the

capacity of a pair of resident buffers, which would stall the wave-streaming discipline. Saga resolves this by extending residency off-device through pinned host memory. Each of the two layer buffers is paired with a pinned DRAM spill buffer. Overflow states are written to the spill buffer as a layer is produced and read back as it is consumed, and the spill buffers alternate in step with the GPU buffers, so producer and consumer never contend.

Because the host memory is pinned, the spill copies overlap with kernel execution rather than stalling it. In effect, the layer is implemented as a logically contiguous staging area whose physical storage spans VRAM and DRAM, with the split between the two determined at run time by the size of the producing layer. A workload whose layers fit within device memory incurs no overflow traffic, and one that overflows incurs it only for the states that do not fit.

Adaptive CPU fallback for narrow layers. The analysis typically opens and closes with layers containing only a handful of states. For these layers, two effects hinder GPU execution. First, kernel-launch overheads dominate the actual computation. Second, available warp-level parallelism is severely underutilized. Consequently, a modest CPU implementation can process these layers even faster than the fixed cost of launching GPU kernels.

Saga monitors the size of each layer and dispatches it to the CPU when it falls below a configurable threshold. Its buffer is copied to host memory to process the layers of expansion and merge, with the result copied back before the next GPU-dispatched layer begins. This is a workload-routing decision informed by the cost model of each architecture: wide layers justify the fixed launch cost and benefit from massive parallelism, while narrow layers are strictly better served on the CPU. Saga thus assigns each layer to the silicon best matched to its size, and neither architecture stalls on the other.

V. Parallel Expansion: Eligibility and Creation

The expansion step of the SAG, characterized in §II, transforms a layer $\mathcal{L}$ of system states into a multiset $\mathcal{L}'$ of successor states. Two structural properties of this transformation, shared by every variant of the SAG, admit a parallel decomposition that is the subject of this section.

For any state $s \in \mathcal{L}$, the set of candidate jobs that may legally be dispatched next is determined entirely by s and the workload parameters. It does not depend on any other state in $\mathcal{L}$. For any feasible pair (s, j), the successor state s' is a function of s, j, and the workload parameters. The expansion therefore separates into a pipeline. An *eligibility* stage maps each s to its set of feasible pairs (s, j). A *creation* stage then maps each feasible pair to its successor.

This decomposition is the backbone of the expansion framework. Each stage is realized as a GPU-resident kernel. The two kernels communicate through a single device-side counter, written by the eligibility kernel and read by the creation kernel, so that no host-device round trip is incurred between them (Figure 3). The remainder of this section formalizes the work

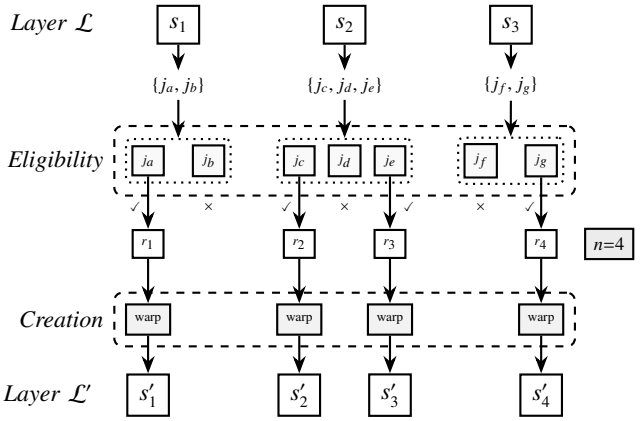

Fig. 3: The expansion pipeline. Each state s in $\mathcal{L}$ has a candidate list of jobs that could be dispatched next, e.g., $\{j_a, j_b\}$ for s_1. The eligibility kernel runs one warp per state; inside each warp (dotted boundary), one thread examines one candidate. Each thread whose candidate passes the eligibility conditions (✓) emits a record r_k. The creation kernel reads the count n, runs one warp per record, and writes a successor to $\mathcal{L}'$.

mapping at each stage, states the abstract parallel primitives required, and establishes correctness. What constitutes eligibility and what constructs a successor differ across the variants of the lineage [1, 2, 4, 5]; the parallelization developed in this section applies uniformly to each.

Work mapping. §III showed that the natural thread-per-state assignment collapses under the variance in candidate-set size across a layer. At the layer granularity, different states admit different numbers of ready candidates. At the per-candidate granularity, different candidates incur different numbers of predicate evaluations. A mapping that binds a single thread to either unit of work exposes the full variance as divergence.

We avoid this by assigning one thread group per unit of work. In the eligibility stage, a single group processes one state. Its threads cooperatively iterate over the candidate list, evaluate each predicate in lock step, and combine per-thread contributions via thread-group reductions to share the cross-candidate information the start-time rule consumes. In the creation stage, a single group processes one feasible pair. The threads cooperatively perform the multi-component successor construction, which includes updates to the per-core availability intervals, pruning of the live finish-time set, and insertion of the new finish-time record.

The mapping rests on three primitives that the thread group must expose: a parallel one-bit reduction across the group (a "ballot"), a parallel minimum reduction, and a broadcast from one thread to the rest. Modern GPUs provide these directly at the warp level; we describe the remainder of this section in terms of GPU warps, which is the realization used in our experimental evaluation. The cross-warp aggregation of the per-job best- and worst-case response-time bounds, BCRT[j] and WCRT[j], additionally uses global atomic minimum and

maximum, a hardware primitive distinct from these group-level reductions.

Eligibility. The eligibility kernel reads the input layer and produces a dense stream of eligibility records. Each record carries a state index, a candidate job index, and the start-time bounds $[s_{\min}, s_{\max}]$ of the pair. The thread group assigned to state s proceeds in two sub-phases.

In the first sub-phase, the group constructs the ready set R of candidates surviving the variant-specific structural eligibility predicate at s. For each $j \in R$, the group computes the per-candidate intermediate that the $s_{\max}$ rule of the variant consumes—the latest moment at which the scheduler could observe j becoming ready under any execution scenario consistent with s. These intermediates are shared across the threads of the group via warp-level reductions, supplying the cross-candidate information that the priority-based start-time computation of the next sub-phase requires.

In the second sub-phase, the group revisits each $j \in R$ and evaluates the start-time interval $[s_{\min}(j), s_{\max}(j)]$ under the priority rule of the scheduler. Where the interval is non-empty, an eligibility record is appended to the output stream by an atomic increment of a shared counter. Many candidates can be ruled out by a cheaper structural test before any start-time interval is computed; such pre-filtering is the subject of §VII.

Creation. The creation kernel consumes the eligibility stream. The launch geometry is determined by the actual count from the device-side counter updated by the eligibility kernel. No host-device synchronization therefore separates the two stages within a wave.

For each eligibility record (s, j), the assigned thread group constructs the three fields of the successor state s' cooperatively. The dispatched set D' is obtained by adding j to a copy of D. The live finish-time set F' is extended with the finish-time bounds of j and pruned: a previously dispatched entry is retired from F' when its finish-time can no longer constrain a later dispatch, a test evaluated by a parallel reduction across the group. The per-core availability A' is obtained by inserting the finish-time interval of the pair into the sorted list of core intervals, and by removing dominated cores in the same pass.

Two side effects accompany creation across the lineage. The per-job response-time bounds BCRT[j] and WCRT[j] are updated by global atomic minimum and maximum against the contribution of the pair. The aggregate bounds at the end of the analysis are therefore the pointwise extrema over every successor ever produced. The successor itself is appended to the output buffer of the wave by an atomic increment of a shared counter. The group then stripes its three fields into the reserved byte range.

Equivalence. We establish the equivalence between the two-stage parallel composition and the sequential SAG expansion by demonstrating that both produce the exact same multi-set of successor states. Fundamentally, this equivalence holds because the parallel architecture systematically eliminates the three primary hazards of parallel state-space exploration: data races on shared memory, non-deterministic thread interleaving, and lost or duplicated work.

First, to prevent data races, the pipeline strictly localizes state creation. The sequential successor function explicitly depends only on the localized record (s, j) and static workload parameters. By assigning a distinct warp to each feasible pair, each warp computes its successor independently. Because there is no cross-warp dependency or shared mutable structure during the creation stage, concurrent warps cannot race.

Second, the architecture immunizes kernel-internal results against non-deterministic thread interleaving. When a warp constructs a successor state, its threads cooperate to distribute tasks, such as pruning the finish-time set or updating availability intervals. Because these cooperative operations rely strictly on commutative and associative primitives (namely minimum, maximum, set union, and bitwise OR), the evaluated successor fields remain mathematically invariant under any arbitrary thread execution schedule.

Finally, the pipeline guarantees strict emission injectivity to ensure no work is dropped or duplicated. The eligibility kernel exhaustively evaluates every (s, j) combination, reserving output slots strictly for feasible pairs via atomic counter increments. Because this atomic operation consistently returns a distinct index to every concurrent caller, distinct feasible pairs are mapped to strictly unique positions in the output stream. The creation kernel subsequently processes this exact, collision-free set of records, guaranteeing that the parallel expansion considers the exact same feasible pairs as the sequential baseline and emits every result without loss.

VI. Parallel Merge: Sort-Based Decomposition

The merge phase consolidates the states produced by the parallel expansion of §V into a smaller layer by collapsing compatible states into a single abstract representative. The compatibility conditions, defined uniformly across the SAG lineage in §II, are (C1) equality of the dispatched set D, (C2) pairwise overlap of the per-core availability intervals, and (C3) pairwise overlap of the intervals in the live finish-time set F; we write $s_1 \sim s_2$ when all three hold. Two states satisfying all three may be replaced by the component-wise union of their interval fields, and the resulting *merged* state is sound for every execution scenario abstracted by either input.

§III identified the sequential hash-map formulation of this phase as the hardest obstacle to GPU parallelization. The slot assignment of the k-th state depends on which of the preceding $k-1$ states were merged together and on the interval bounds induced by those earlier merges. No data-parallel schedule can respect this dependence without reintroducing a serial barrier. The framework of this section resolves the obstacle by replacing the hash-map formulation with a *sort-based decomposition* whose every stage is data-parallel.

The decomposition has three stages. A *sorting* stage arranges the states contiguously by their dispatched set, so that states sharing a D (and thus satisfying C1) occupy adjacent positions. A *grouping* stage detects the boundaries between distinct D values and partitions the layer into per-D groups.

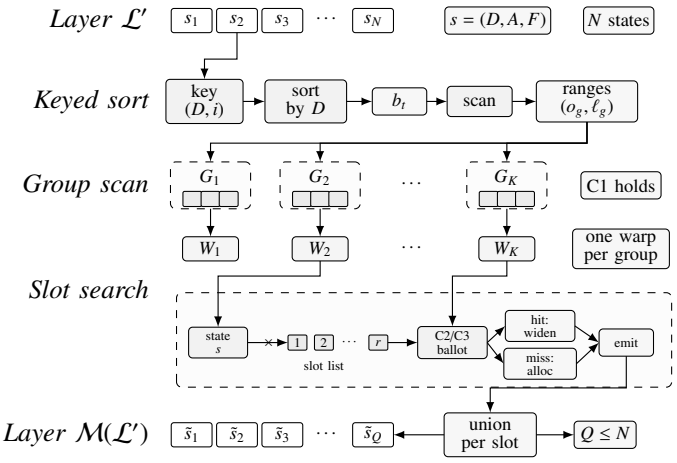

Fig. 4: The sort-based merge pipeline. The post-expansion layer $\mathcal{L}'$ is converted to (D, index) key–value pairs, sorted by the dispatched set D, and scanned using boundary flags (b_i) to recover contiguous per-D groups, represented by their start offsets and lengths (o_g, ℓ_g) (sorting enforces C1). Each group is assigned to a warp, which maintains the slot list in shared memory; a candidate state is tested against all active slots in parallel under C2 and C3, then either widens a compatible slot or allocates a new one. The emitted slot representatives form the merged layer $\mathcal{M}(\mathcal{L}')$.

A *slot-search* stage partitions each group into slots under the residual conditions C2 and C3, and collapses each slot to a single merged state. The resulting pipeline is summarized in Figure 4.

Sorting and group boundary detection. Sorting a layer of states by their dispatched set is a standard parallel task. Each state is represented by a (D, index) key-value pair. For variants of the lineage in which D fits in a single machine word, parallel radix sort is the natural choice. For variants with larger D, a parallel comparison-based sort is invoked with a multi-word key comparator. In both cases the sort preserves the original indices, so that the full state records can be accessed indirectly through the sorted index array. Our implementation uses GPU-resident radix sort and merge sort routines, but any parallel sort with GPU-resident throughput is sufficient for this stage.

Once states are sorted by D, group boundaries are detected by a parallel comparison kernel. Thread i emits a boundary flag if the D of the i-th sorted state differs from that of the $(i-1)$-th. A parallel exclusive scan then converts the boundary flag array into a per-state group identifier, and a compaction step collects the group start positions and sizes. These are all standard device-parallel primitives. After this stage, the layer is represented as a list of groups, each containing all states that share a single D and is ready for the slot search.

Parallel slot search. Within a group, all states satisfy C1 by construction. The merge phase therefore reduces to the residual task of partitioning the group into slots under conditions C2 and C3, and collapsing each slot to a representative. The sequential hash-map formulation processes the group one state at a time and queries an incrementally maintained slot list. In practice, the slot list of a group is typically small, substantially fewer than 32 in common workloads. This admits a fully parallel alternative: the compatibility of a candidate state against *all* slots currently in the list can be tested concurrently, with one thread assigned to each slot.

We realize this observation by assigning one thread group to each per-D group. The thread group maintains the slot list in shared memory. For each incoming state in the group, the threads evaluate the compatibility predicate (C2 and C3) between the incoming state and their assigned slots in parallel. A one-bit parallel reduction across the group identifies whether any slot is compatible and, if so, which one to merge into. If a compatible slot exists, the winning thread cooperatively widens the interval bounds of that slot with the bounds of the incoming state. If no slot is compatible, a new slot is allocated at the end of the list and the incoming state is copied into it. For groups whose slot list exceeds the thread-group width, the search proceeds in multiple rounds, with the candidate broadcast to each round and the parallel compatibility test repeated.

The decomposition transforms what the sequential hash-map renders as an inherently serial walk into a data-parallel predicate evaluation. The primitives required are exactly those used by the expansion kernels of §V: a parallel one-bit reduction, a parallel minimum, and a broadcast. On GPU hardware one warp per group is a natural fit, since typical slot counts fall well within warp width. Our implementation uses this warp-per-group mapping.

Safeness. Unlike expansion, where §V established equivalence with the sequential algorithm, the merge phase is order-dependent even in its original sequential formulation. We accordingly argue *soundness* rather than equivalence: the merged layer abstracts every scenario represented by the input layer, though the specific slot partition depends on the ordering used.

Definition 1 (Scenario abstraction)**.** *A state s* represents *an execution scenario ξ if the interval-valued parameters of ξ lie within the interval bounds of s and the dispatch prefix of ξ matches the dispatched set D of s. A layer $\mathcal{L}^*$* soundly abstracts *a layer $\mathcal{L}$ if every scenario represented by any state in $\mathcal{L}$ is represented by some state in $\mathcal{L}^*$.*

Lemma 1 (Soundness of merge operator)**.** *Let s_1, s_2 be two states satisfying C1, C2, and C3, and let $s^* = \text{merge}(s_1, s_2)$ denote their component-wise union. Every scenario represented by s_1 and every scenario represented by s_2 is represented by s^*.*

Proof. Condition C1 gives $D_1 = D_2$, so s^* inherits the common dispatched set $D^* = D_1 = D_2$, preserving the dispatch-prefix match required by Definition 1. Each interval-valued field of s^* is, by construction, the union of the corresponding fields of s_1 and s_2, so every interval bound of s^* is a

component-wise superset of the corresponding bound in either input. The set of scenarios represented by a state is monotone in its interval bounds: widening can only include additional scenarios. Every scenario represented by s_1 or s_2 is therefore represented by s^*. □

Lemma 1 is inherited from the sequential SAG literature [1, 2, 4, 5]; we restate it because the soundness of our decomposition rests on it.

Theorem 1 (Soundness of the parallel merge)**.** *Let $\mathcal{L}'$ be a post-expansion layer, and let $\mathcal{M}(\mathcal{L}')$ be the layer produced by the sort-based decomposition of this section. Then $\mathcal{M}(\mathcal{L}')$ soundly abstracts $\mathcal{L}'$.*

Proof. The decomposition partitions $\mathcal{L}'$ into per-D groups via sorting and boundary detection (which reorder states but do not alter them), then into slots $S_1, \ldots, S_k$ via the parallel slot search; $\mathcal{M}(\mathcal{L}')$ consists of one representative $\tilde{s}_i$ per slot, the iterated component-wise union of the states in S_i.

Fix $s \in \mathcal{L}'$ and let S_i be the slot containing s. Every pairwise merge within S_i combines a state with the current slot representative under C1, C2, C3; Lemma 1 applies at each step, so by induction every scenario represented by any state in S_i — including s — is represented by $\tilde{s}_i \in \mathcal{M}(\mathcal{L}')$. This holds for every $s \in \mathcal{L}'$; the slot-search ordering affects only the partition $\{S_1, \ldots, S_k\}$, not whether Lemma 1 applies at each pairwise merge. □

Remark 1 (Non-transitivity of the compatibility relation)**.** *The compatibility relation $\sim$ defined by C1, C2, C3 is reflexive and symmetric but not transitive (Figure 5), the same structure that makes state minimization of incompletely specified finite-state machines NP-complete [13].*

Non-transitivity leaves the merged layer an artefact of the order in which pairs are attempted. It is natural to ask whether some ordering is distinguished, an optimal one that every implementation, sequential or parallel, would be obliged to obey. Theorem 2 rules this out for any polynomial-time analysis.

Theorem 2 (Hardness of minimum-size merge ordering)**.** *Minimizing the number of states of a SAG over its merge orderings is NP-hard, even when merging does not widen interval bounds.*

Proof. We first establish hardness for a single layer, then lift it to the entire graph. We characterize the minimum graph-theoretically, then reduce from minimum clique cover. Let $G_\sim(\mathcal{L}')$, the *compatibility graph* of $\mathcal{L}'$, be the simple undirected graph with vertex set $\mathcal{L}'$ and edges $\{\{s_i, s_j\} : s_i \sim s_j\}$, and let $\mathcal{M}_\pi(\mathcal{L}')$ denote the layer produced by a merge ordering π run until no compatible pair remains. We work under the relaxation in which the merge operation does not widen interval bounds, so that admission of a state into a slot requires pairwise compatibility with every existing member of that slot, rather than compatibility with a widened representative. Under this relaxation, every slot of $\mathcal{M}_\pi(\mathcal{L}')$ is a pairwise-compatible subset of $\mathcal{L}'$, hence a clique of $G_\sim(\mathcal{L}')$; conversely, any clique cover of $G_\sim(\mathcal{L}')$ is realized by an ordering that processes one clique at a time. Therefore $\min_\pi |\mathcal{M}_\pi(\mathcal{L}')| = \theta(G_\sim(\mathcal{L}'))$, the clique cover number, equivalently $\chi(\overline{G_\sim(\mathcal{L}')})$, the chromatic number of the complement.

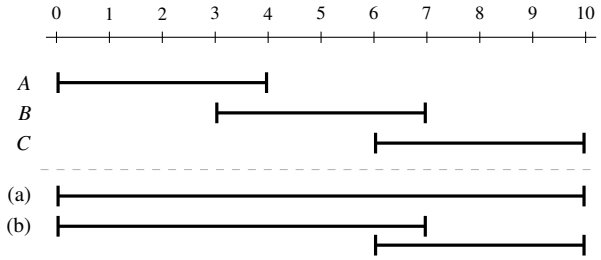

Fig. 5: **Order-dependence of state merging under non-transitive compatibility.** Consider three states with availability intervals $A = [0, 4]$, $B = [3, 7]$, and $C = [6, 10]$ in the same per-D group on a single core (making condition C3 trivially satisfied). (a) Processing in the sequence A, B, C yields a single merged slot: state B merges into the slot opened by A, widening its interval to $[0, 7]$, which subsequently overlaps with and absorbs C to $[0, 10]$. (b) Processing in the sequence A, C, B yields two distinct slots: state C lacks overlap with the initial interval $[0, 4]$ of A and opens a new slot $[6, 10]$; state B overlaps with both existing slots but merges into the first compatible slot it evaluates, widening the slot opened by A to $[0, 7]$.

We reduce minimum clique cover, which is NP-hard on arbitrary simple graphs [14], to the optimal-ordering problem. Given an arbitrary graph $H = (V_H, E_H)$, we construct a layer $\mathcal{L}_H$ such that $G_\sim(\mathcal{L}_H) \cong H$. Encode each vertex $v \in V_H$ as a state s_v, and assign all states the same dispatched set so that C1 holds among every pair. For each non-edge $\{u, v\} \notin E_H$, reserve a fresh dimension d_{uv} of the compatibility test (a core of C2 or an F entry of C3) in which s_u and s_v are assigned disjoint intervals, while every other state pair is assigned overlapping intervals in d_{uv}. Remaining dimensions are assigned universally compatible bounds. The total number of dimensions required is $O(|V_H|^2)$, which the SAG framework accommodates when the workload exposes sufficient cores or F entries. By construction, $s_u \sim s_v$ in $\mathcal{L}_H$ iff $\{u, v\} \in E_H$, so $G_\sim(\mathcal{L}_H) \cong H$ and $\min_\pi |\mathcal{M}_\pi(\mathcal{L}_H)| = \theta(H)$. An oracle for the optimal-ordering problem therefore solves minimum clique cover, and NP-hardness carries over.

Now place $\mathcal{L}_H$ in the terminal layer, which has no successors, and let the earlier layers be free of compatible pairs, so that they contribute a constant Σ. The graph size is then $\Sigma + |\mathcal{M}_\pi(\mathcal{L}_H)|$, and the hardness extends to the entire graph. □

Theorem 2 places optimal merge ordering in the broader NP-hard family of axis-parallel box partitioning problems [15]. Every SAG implementation, including the sequential hash-map formulation [1, 2, 4, 5], is therefore a polynomial-time greedy algorithm under a fixed ordering, not an optimization procedure, and no optimal merge ordering is available at polynomial

cost for a parallel implementation to obey. Empirically, on the tasksets of our evaluation (§VIII), the observed relative difference in per-job worst-case response times between the parallel and sequential merges is bounded by 0.2%.

VII. Heuristic Pre-Filtering

The eligibility kernel of §V runs interval-arithmetic checks for every (state, candidate) pair in a wave. Many pairs are provably non-eligible by structural reasoning alone, and the SAG literature provides sufficient conditions for early rejection: predecessor-readiness for the DAG and event-driven variants [4, 5] and priority-order dominance [7]. Our contribution is not a new condition but a uniform GPU representation that turns any such condition into a single-cycle subset test, plus the SIMT-aware procedure for applying it.

Every such filter takes the same form: a per-job, n-wide set $F(j)$ tested against the dispatched set D_s of the parent state by the inclusion $F(j) \subseteq D_s$, rejecting the candidate unless the test passes. The two cited filters are instances of this form, with $F(j)$ the precedence-ancestor closure and the priority-order dominance closure, respectively; any condition expressible as "a specified set of jobs must be dispatched before j" admits the same form. The abstraction is (1) **uniform**: as all filters share one code path; (2) **composable**: meaning a candidate survives only if every installed F passes, while the evaluation order itself remains immaterial; and (3) **decoupled**: since the kernel sees only the bits, whereas the theoretical soundness of each filter has been established in prior work.

The form aligns with SIMT execution: each thread tests its candidate against the group-shared D_s and rejects early, so only survivors enter the interval arithmetic of §V. The benefits are concrete: (1) **reduced work**: as downstream cost scales with the survivor count rather than the nominal candidate count; (2) **reduced divergence**: because the surviving candidates are more homogeneous, recovering throughput that the thread-per-state mapping of §III loses to divergence; and (3) **coalesced access**: since the F tables are read-only and row-major, allowing one cache line to serve the test of every thread for a candidate batch.

A new sound condition in this form costs one n-wide table and one extra subset test per candidate; the kernel itself does not change. Future refinements to the underlying theory compose into the pre-filtering stage without modification of the framework.

VIII. Evaluation

We implemented Saga as a CPU–GPU framework. See Algorithm 1 for a quick overview. The CPU parses CSV inputs, selects the SAG variant, allocates device and pinned-host buffers, and drives the layer loop. Within each layer, the CPU uses two CUDA streams and two wave-buffer sets in a ping-pong manner: while one stream executes kernels for the current wave, the other stream stages the next wave. GPU kernels and CUB primitives then perform filtering, expansion, bound updates, sorting, grouping, and merging.

Algorithm 1: Main Architectural Flow of Saga

```
Input: jobs.csv, jobsprec.csv, cores m, variant V
Output: Schedulability verdict and BCRT/WCRT bounds
CPU: parse inputs, select variant, allocate buffers;
CPU: create two CUDA streams and two wave-buffer sets;
CPU: initialize the first SAG layer on the GPU;
while current layer is nonempty do
    CPU: split the layer into GPU-sized waves;
    CPU: preload the first wave into one buffer;
    foreach wave do
        CPU: use one stream/buffer for the current wave;
        CPU: stage the next wave on the other stream/buffer;
        GPU: filter candidates and expand successor states;
        GPU: update BCRT/WCRT bounds and status counters;
        GPU/CUB: sort, group, and merge states by key;
        CPU: read counters after the current stream completes;
    end
    if scratch or layer buffers exceed capacity then
        CPU/GPU: spill overflow states through pinned host memory;
    end
    CPU/GPU: consolidate wave outputs and swap layer buffers;
end
return verdict and final BCRT/WCRT bounds;
```

We evaluate Saga against SAG-org EDD analyzer of Srinivasan et al. [5] — the most computationally demanding member of the SAG lineage. Under shared workload parameters, the EDD variant's analysis median wall time exceeds the other three published SAG variants [1, 3, 4] by at least two orders of magnitude. The evaluation spans three axes: *speedup*, *accuracy*, and *cross-platform portability*. To assess these metrics, synthetic task sets are generated following the UUniFast-derived methodology of the original SAG papers.

Experimental setup. Table I summarizes the experimental setup, including the hardware platforms, baseline tool, and workload generation parameters. The framework runs the same binary across all three GPUs with no architecture-specific code paths; only the GPU device changes between runs.

TABLE I: Hardware platforms, baseline, and workload generator configuration.

Hardware and software setup	
Primary GPU	NVIDIA B200 (192 GB HBM3e)
Comparison GPUs	NVIDIA H100 (80 GB HBM3); NVIDIA A100 (40 GB HBM2)
GPU driver	CUDA 12.5
Host	Intel Xeon Platinum 8592, 4 TB DDR5
CPU baseline	SAG-org (`nptest`), EDD analyzer of Srinivasan et al .[5]
Workload generator: UUniFast	
Tasks per set (n)	$\{16, 20, 24, 28, 32, 36, 40, 44, 48\}$
Cores (m)	$n/4$ (4:1 ratio)
Total utilization (U')	$\{20, 40, 60, 80\}\%$
BCET / WCET ratio (β)	0.5
Segments per task (S)	5
Deadline type	implicit ($D = T$)
Priority assignment	deadline-monotonic
Replicates	25 fresh seeds per (n, m, U') point (100 task sets per n)

Wall-time and speedup on B200. In Fig. 6, the `nptest` median follows an exponential trend, exceeding 50 s at $n = 16$ and saturating the cap at $n = 32$. The B200 median exhibits

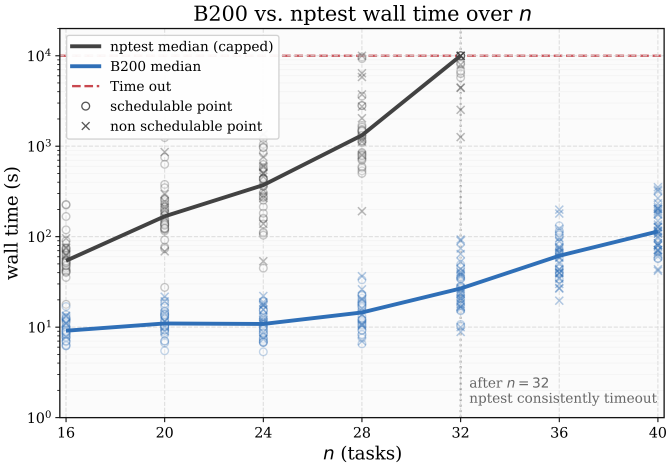

Fig. 6: Wall times of the framework on B200 and of `nptest`, over $n \in [16, 40]$. Markers summarize 100 task sets per n, with circles indicating schedulable task sets and crosses non-schedulable; solid lines show per-n medians and the dashed line is the 10,000 s harness cap.

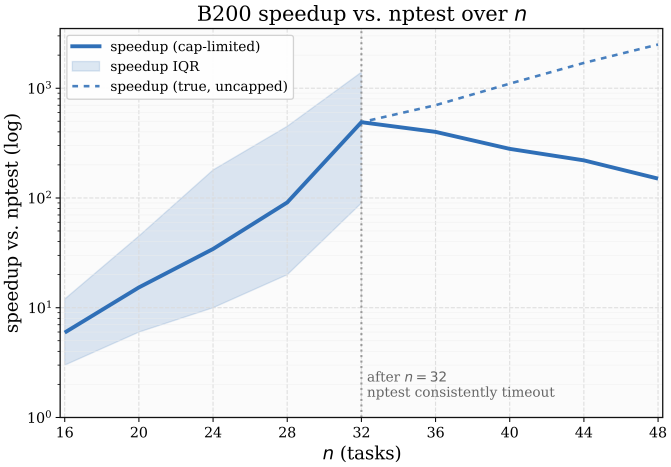

Fig. 7: B200 speedup over `nptest`, log scale. Solid: cap-bounded measured ratio. Shaded band: inter-quartile range. Dashed: extrapolated ratio with the cap removed.

a piecewise behavior: a fixed-cost regime, driven by CUDA-context, kernel-launch, and per-layer sort overheads, persists through $n = 28$, followed by an exponential growth regime reaching ~ 28 s at $n = 32$ and ~ 105 s at $n = 40$. The ratio of these two trends is the cap-bounded median speedup (Fig. 7), which increases through $n \in [16, 32]$ to $\sim 15\times$ at $n = 20$, $\sim 34\times$ at $n = 24$, $\sim 91\times$ at $n = 28$, and $\sim 470\times$ at $n = 32$. Past $n = 32$, `nptest` consistently times out within the 10,000 s harness cap on every task set in the grid; direct wall-time comparison is no longer meaningful. The solid curve declines past this point only because its numerator is fixed at the cap while the B200 denominator continues to grow. The dashed extrapolation extends `nptest`'s pre-cap exponential trend beyond the harness ceiling and divides by the measured

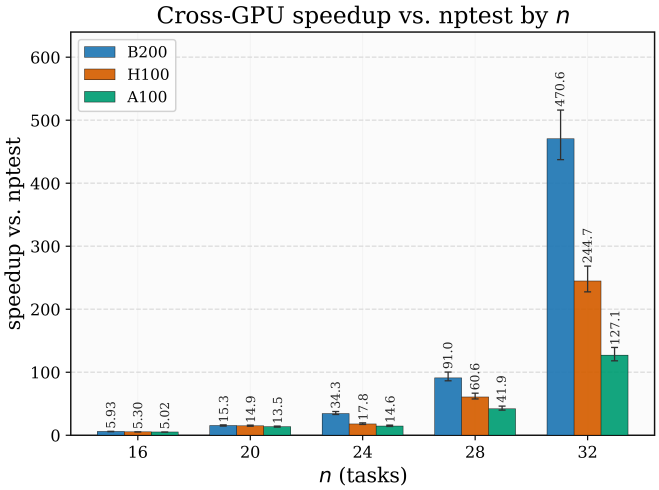

Fig. 8: Median per-task-set speedup over `nptest` on B200, H100, and A100 for $n \in [16, 32]$; error bars show the inter-quartile ranges of these speedups.

B200 wall, exceeding three orders of magnitude near $n = 40$ and reaching approximately 2,100× at $n = 48$.

TABLE II: Per-job comparison with `nptest`. "Both SCHED" is the subset of the 100 task sets at each n in which both tools converge to SCHED within the `nptest` harness cap. "Avg WCRT/BCRT" is the mean absolute per-job relative deviation $|X_j^{\text{GPU}} - X_j^{\text{nptest}}|/X_j^{\text{nptest}}$, averaged over every such job.

n	Sets	Both SCHED	avg WCRT (%)	avg BCRT (%)
16	100	82	0.05	0.02
20	100	71	0.09	0.04
24	100	46	0.14	0.07
28	100	29	0.17	0.09
32	100	14	0.19	0.11

Accuracy against the baseline. We compare per-job WCRT and BCRT bounds against `nptest` across all configurations within the baseline's tractability limit, namely $n \in \{16, 20, 24, 28, 32\}$.

As detailed in Table II, across the 242 task sets in which both tools converge to SCHED (25,480 jobs total), the mean per-job WCRT and BCRT deviations grow mildly with n, from 0.05% to 0.19% for WCRT and from 0.02% to 0.11% for BCRT. The deviation reflects the parallel and sequential algorithms traversing different greedy merge orders under a non-transitive compatibility relation; the speedup figures below therefore reflect a change of execution, not of abstraction.

Cross-GPU generalization. At $n = 16$ the three medians (Fig. 8) are 5.02× on A100, 5.30× on H100, and 5.93× on B200. At this small workload scale, execution time is strictly bounded by the CPU fallback for narrow layers and the fixed CUDA-context and kernel-launch costs for GPU-dispatched layers, which together mask the underlying architectural differences across platforms. The B200/A100 ratio then grows

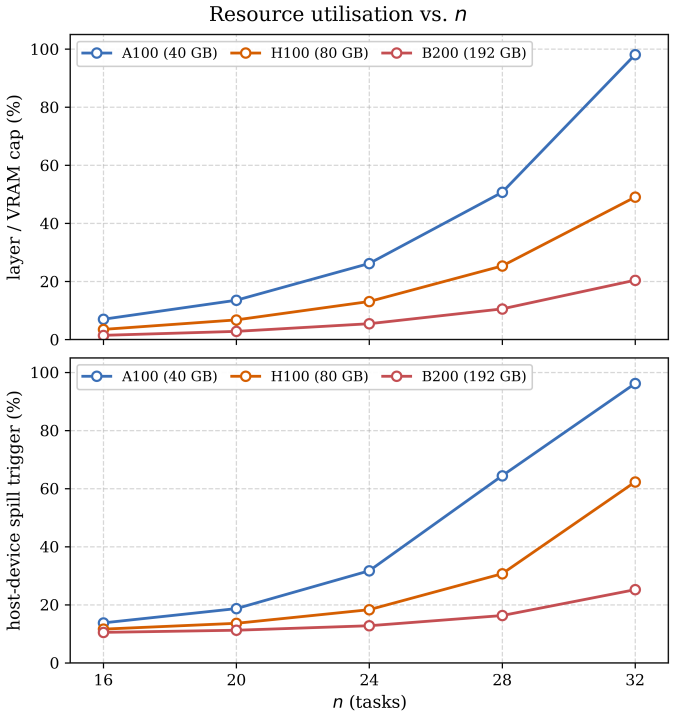

Fig. 9: Resource utilization for $n \in \{16, 20, 24, 28, 32\}$. Top: peak per-layer footprint relative to the per-buffer VRAM cap of each platform. Bottom: fraction of waves spilling to pinned host memory.

from 1.13× at $n = 20$ to 2.35× at $n = 24$, as older platforms enter their exponential regime before B200. At $n = 32$ the medians reach their maxima of 127× on A100, 244× on H100, and 470× on B200.

Resource utilization. In Fig. 9, per-layer footprint (top panel) depends on the workload and not on the hardware; the separation between curves reflects cap differences alone. A100 (40 GB) reaches its cap at $n = 32$, H100 (80 GB) reaches $\sim 50\%$ of its cap at the same n, and B200 (192 GB) remains below 25% throughout. The spill trigger rate (bottom panel) follows the footprint-to-cap pressure: A100 crosses 50% at $n = 28$ and approaches 100% at $n = 32$, H100 reaches $\sim 60\%$ at $n = 32$, and B200 remains below 30% throughout. Consequently, the performance bottleneck of the SAG analysis fundamentally shifts: it is no longer bound by compute time, but rather constrained by VRAM capacity and host-device memory bandwidth.

IX. Related Work

The only direct precedent for our work dates back to 2013, when Ahmed et al. [16] parallelized EDF feasibility checking. Saga advances this decade-old baseline through a new algorithmic foundation and a much broader scope. Specifically, our framework accelerates the SAG state-space abstraction rather than relying on direct EDF enumeration. Furthermore, it moves beyond simple EDF feasibility to cover the full SAG lineage [1, 2, 4, 5].

The closest framework analog is Gharaibeh et al. [17], who introduced a hybrid CPU-GPU graph processing system that partitions workload between the two architectures by static graph properties. Saga shares the heterogeneous-dispatch principle but differs in domain (accurate schedulability verification rather than general graph algorithms), adds a VRAM-overflow spill path for layers exceeding device memory, and routes by dynamic layer size rather than static partitioning. StarPU [18] is a generic heterogeneous CPU-GPU task-scheduling runtime; Saga instead uses a workload-specific size-based dispatch tuned to the narrow-layer behavior of SAG.

Gunrock [19] is the canonical GPU-only graph framework, built around three data-parallel primitives: advance, filter, and compute. These primitives are precisely what the irregular per-state expansion and order-dependent merge of SAG resist (§III), motivating the heterogeneous design Saga adopts instead.

The closest domain analog is GPUexplore [20], an explicit-state model checker performing on-the-fly exploration on GPUs. Like SAG construction, it employs an exhaustive search combined with pruning. However, GPUexplore fundamentally differs in two ways: it lacks CPU-GPU heterogeneity, and it operates without interval-valued state merging (the exact abstraction that gives SAG its compactness).

X. Conclusion and Future Work

This paper introduced Saga, a GPU-accelerated computational framework for exploration-merging schedulability analyses, developed concretely for the Schedule Abstraction Graph (SAG) lineage. Saga overcomes the sequential bottlenecks of state-space exploration through a novel architecture that combines wave-based streaming orchestration, warp-cooperative parallel expansion, a sort-based merge decomposition, and extensible pre-filtering. Consequently, it delivers a massive leap in execution efficiency over state-of-the-art CPU tools.

The case for hardware acceleration was deliberately framed through the hardest schedulability paradigm to parallelize. Irregular graph algorithms with order-dependent merging are widely recognized as one of the most hostile classes of workloads for data-parallel hardware. Saga successfully tames this irregularity not to replace, but to synergize with the algorithmic achievements in the real-time systems community. Saga fully inherits the precision and soundness guarantees of its sequential counterparts, ensuring that past and future algorithmic refinements compose with this hardware acceleration directly.

By transforming intractable offline analysis into near-instantaneous verification, Saga shifts accurate analysis from a last-resort certification step into a routine design-time utility. Within the real-time domain, Saga unlocks more expressive models, such as cache-aware SAG variants and coupled task-resource models. Previously limited by runtime constraints, these models are now practical candidates for exact treatment. Furthermore, this computational paradigm generalizes beyond real-time scheduling. Timed-automata engines like UPPAAL

and TChecker face identical sequential exploration bottlenecks, and can now directly leverage the abstract operators of SAGA to achieve similar scalability.

Over the past decade, sustained algorithmic progress has brought accurate analysis to an unprecedented level of expressiveness and precision. Yet, such verification can no longer afford to be constrained by the architectural plateau of single-core CPUs. It is time to advance toward a paradigm of algorithm-hardware co-design. By bridging this gap, SAGA opens the next frontier of research and invites synergistic collaboration between the real-time and HPC communities, revealing the true potential of expressive theoretical models when accelerated by the vast parallel hardware infrastructures already at our disposal.

XI. Acknowledgments

Responsible use of generative AI. ChatGPT (OpenAI) and Claude were used only as a language-editing aid to improve the grammar, phrasing, and readability of author-prepared draft text throughout the manuscript. Claude was also used to generate the tikz representation of Figures 3 and 5, and to provide general coding assistance.

References

[1] M. Nasri and B. B. Brandenburg, "An exact and sustainable analysis of non-preemptive scheduling," in *2017 IEEE Real-Time Systems Symposium (RTSS)*, IEEE, 2017, pp. 12–23.

[2] M. Nasri, G. Nelissen, and B. B. Brandenburg, "A response-time analysis for non-preemptive job sets under global scheduling," in *30th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 106, Schloss Dagstuhl, 2018, 9:1–9:23. DOI: 10.4230/LIPIcs.ECRTS.2018.9.

[3] G. Nelissen, J. Marcè i Igual, and M. Nasri, "Response-time analysis for non-preemptive periodic moldable gang tasks," in *34th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 231, Schloss Dagstuhl, 2022, 12:1–12:22. DOI: 10.4230/LIPIcs.ECRTS.2022.12.

[4] M. Nasri, G. Nelissen, and B. B. Brandenburg, "Response-time analysis of limited-preemptive parallel DAG tasks under global scheduling," in *31st Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 133, Schloss Dagstuhl, 2019, 21:1–21:23. DOI: 10.4230/LIPIcs.ECRTS.2019.21.

[5] S. Srinivasan, M. Günzel, and G. Nelissen, "Response-time analysis for limited-preemptive self-suspending and event-driven delay-induced tasks," in *2024 IEEE Real-Time Systems Symposium (RTSS)*, IEEE, 2024, pp. 29–42.

[6] S. Srinivasan, G. Nelissen, R. J. Bril, and N. Meratnia, "Analysis of TSN time-aware shapers using schedule abstraction graphs," in *36th Euromicro Conference on Real-Time Systems (ECRTS)*, ser. LIPIcs, vol. 298, Schloss Dagstuhl, 2024, 16:1–16:24. DOI: 10.4230/LIPIcs.ECRTS.2024.16.

[7] S. Ranjha, P. Gohari, G. Nelissen, and M. Nasri, "Partial-order reduction in reachability-based response-time analyses of limited-preemptive dag tasks," *Real-Time Systems*, vol. 59, no. 2, pp. 201–255, 2023.

[8] P. Gohari, G. Nelissen, J. Voeten, and M. Nasri, "Leveraging parallelism in global scheduling to improve state space exploration in the sag framework," in *Proceedings of the 32nd International Conference on Real-Time Networks and Systems*, 2024, pp. 70–81.

[9] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997. DOI: 10.1007/s100090050010.

[10] P. Ekberg and W. Yi, "Uniprocessor feasibility of sporadic tasks remains conp-complete under bounded utilization," in *2015 IEEE Real-Time Systems Symposium*, IEEE, 2015, pp. 87–95.

[11] V. Bonifaci, A. Marchetti-Spaccamela, S. Stiller, and A. Wiese, "Feasibility analysis in the sporadic DAG task model," in *25th Euromicro Conference on Real-Time Systems (ECRTS)*, IEEE, 2013, pp. 225–233. DOI: 10.1109/ECRTS.2013.32.

[12] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "NVIDIA Tesla: A unified graphics and computing architecture," *IEEE Micro*, vol. 28, no. 2, pp. 39–55, 2008. DOI: 10.1109/MM.2008.31.

[13] C. P. Pfleeger, "State reduction in incompletely specified finite-state machines," *IEEE Transactions on Computers*, vol. C-22, no. 12, pp. 1099–1102, 1973. DOI: 10.1109/T-C.1973.223655.

[14] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds., New York: Plenum Press, 1972, pp. 85–103.

[15] S. Bereg, S. Cabello, J. M. Díaz-Báñez, P. Pérez-Lantero, C. Seara, and I. Ventura, "The class cover problem with boxes," *Computational Geometry*, vol. 45, no. 7, pp. 324–334, 2012. DOI: 10.1016/j.comgeo.2012.02.002.

[16] M. Ahmed, S. Rampersaud, N. Fisher, D. Grosu, and L. Schwiebert, "GPU-based parallel EDF-schedulability analysis of multi-modal real-time systems," in *Proceedings of the 15th IEEE International Conference on High Performance Computing and Communications (HPCC) and the 11th International Conference on Embedded and Ubiquitous Computing (EUC)*, 2013, pp. 254–263. DOI: 10.1109/HPCC.and.EUC.2013.45.

[17] A. Gharaibeh, L. B. Costa, E. Santos-Neto, and M. Ripeanu, "A yoke of oxen and a thousand chickens for heavy lifting graph processing," in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2012, pp. 345–354.

[18] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, "StarPU: A unified platform for task scheduling on heterogeneous multicore architectures," *Concurrency and Computation: Practice and Experience*, vol. 23, no. 2, pp. 187–198, 2011.

[19] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the GPU," in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2016, pp. 1–12.

[20] A. Wijs and D. Bošnački, "GPUexplore: Many-core on-the-fly state space exploration using GPUs," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Springer, 2014, pp. 233–247.